RV College of Engineering®
Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

MOBILE APPLICATION DEVELOPMENT
MCA221IA
SEE PROJECT BASED LABORATORY

Smart Complaint Management System (SCMS)

submitted by

Pavan Naik	**1RV25MC066**
Prabhava Upadhyaya V	**1RV25MC068**
Prem Kamble	**1RV25MC074**
Pramath Hegde	**1RV25MC070**

under the guidance of

Prof. Chandrani Chakravorty & Prof. Prashanth K

Department of Master of Computer Applications
RV College of Engineering, Bengaluru – 560059
2025-2026

RV College of Engineering®

Mysore Road, RV Vidyaniketan Post,Bengaluru - 560059, Karnataka, India

CERTIFICATE

Certified that the project entitled "**Smart Complaint Management System (SCMS**)" on **Mobile Application Development – MCA221IA** has been carried out by **Pavan Naik (1RV25MC066), Prabhava Upadhyaya V (1RV25MC068), Prem Kamble (1RV25MC074) and Pramath Hegde (1RV25MC070)** who have successfully completed the project for the final SEE Lab Examination, incorporating all concepts of the course conducted by the Department of MCA, RV College of Engineering, Bengaluru.

Internal Guide	**Head of the Department**
Prof. Chandrani Chakravorty	**Dr. Jasmine K S**
Assistant Professor	Associate Professor & Director
Department of MCA	Department of MCA,
RV College of Engineering	RV College of Engineering

External Viva Examination

Name of Examiners	**Signature with Date**
1.	
2.	

Table of Contents

List of Figures

1. Introduction

The Smart Complaint Management System (SCMS) is a mobile-first platform developed to digitise and streamline the way complaints are raised, routed, tracked and resolved within an educational campus. It replaces informal, untracked channels — verbal requests, paper registers, scattered chat groups — with a single, accountable workflow in which every complaint has an owner, a status and a service-level deadline. Within the app, students raise complaints about issues such as plumbing, electrical faults, housekeeping, civil maintenance and IT, attach geotagged photographic evidence, and follow the progress of each complaint from submission to resolution.

SCMS is built as three cooperating services. A Flutter mobile application provides the user interface for all four roles; a Node.js / Express backend exposes the REST API, enforces authentication and authorisation, and runs scheduled background jobs; and a Python (FastAPI) AI microservice augments the workflow with Google Gemini for grammar correction, automatic categorisation and duplicate-complaint detection. The mobile app never contacts the AI service directly — all AI requests are proxied through the Node.js backend, keeping the microservice internal and best-effort. Data is persisted in PostgreSQL, extended with the pgvector extension so that complaint descriptions can be stored as embeddings and compared for similarity.

The system implements role-based access control with four roles — Student, Staff, Student Representative (SR) and Admin. Students submit and rate complaints; SRs review submissions and assign them to the appropriate staff; staff act on and resolve complaints; and admins manage users, departments and analytics. Authentication is through Google OAuth 2.0 restricted to the institutional domain rvce.edu.in, so only verified campus accounts can sign in. Figure 1 shows how these services are arranged.

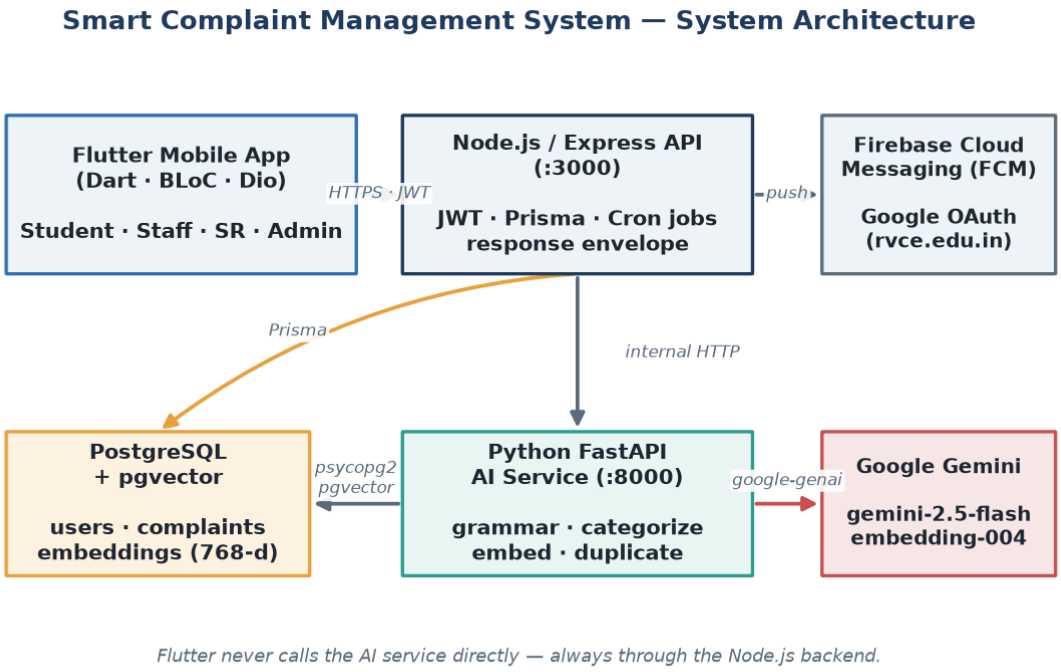

Figure 1: Three-tier system architecture of SCMS

1.2 Overview of Problem Domain

Campuses generate a continuous stream of maintenance and service issues — a leaking tap, a broken projector, a power outage in a lab, an uncleaned corridor. Traditionally these are reported informally and handled without any systematic record. This creates several recurring problems: complaints are lost or forgotten, there is no accountability for who should act, students have no visibility into progress, and the same issue is often reported many times by different people.

The absence of structure also makes prioritisation impossible. Without a severity level or a service-level agreement (SLA), an urgent electrical hazard receives the same attention as a minor cosmetic issue. Manual routing means complaints frequently reach the wrong department, adding delay. And because there is no consolidated data, management cannot see trends, recurring problem areas or staff workload.

SCMS addresses this problem domain by providing a centralised, role-based workflow with enforced accountability. Every complaint is authenticated to a real campus user, routed through a review gate, assigned to a responsible member of staff, tracked against an SLA deadline, and checked for duplicates before it enters the queue. AI assistance improves the quality and consistency of submissions, while analytics give administrators a live view of the campus's complaint landscape.

2. Key Features and Analysis

SCMS combines secure authentication, structured complaint handling, AI augmentation and analytics into a single platform. The major features are described below along with a short analysis of the value each one provides. Figure 2 summarises how these features relate to the platform.

2.1 Role-Based Access Control

The system supports four distinct roles, each backed by a JWT claim (ROLE_USER, ROLE_STAFF, ROLE_SR, ROLE_ADMIN) and guarded on the server:

- Student (User): raises complaints, tracks their status and rates the resolution.
- Staff: sees complaints assigned to them and updates their status through to resolution.
- Student Representative (SR): reviews newly submitted complaints and assigns them to staff.
- Admin: manages users, departments, categories and zones, and views analytics.

Route-level guards ensure that each role can reach only the actions it is permitted to perform, preventing, for example, a student from changing a complaint's status or an unauthenticated request from reaching protected endpoints.

2.2 Secure Authentication with Google OAuth and JWT

Sign-in is exclusively through Google OAuth 2.0, restricted to the rvce.edu.in hosted domain — there is no email/password login. The Google ID token returned to the app is verified on the backend, which additionally enforces an email-domain allowlist before issuing its own short-lived access token and a refresh token. The mobile app stores these tokens in secure storage and transparently refreshes the access token when it expires, so sessions remain both secure and seamless.

2.3 Complaint Submission with Geotagged, Watermarked Photos

When raising a complaint, a student provides a subject, a description, a severity level and an optional location, and may attach up to three photos as evidence. Each photo is stamped with the capture GPS coordinates and a date-time watermark before upload, which discourages the reuse of old or unrelated images and gives staff reliable context about where and when the issue was observed.

2.4 AI Grammar Correction

As the student types the description, the app debounces the input and asks the backend to run a grammar and clarity pass through the AI service (Google Gemini). The corrected text is offered back as a suggestion the student can accept with a single tap. This raises the quality and readability of complaints without forcing the student to rewrite them, which in turn helps staff understand issues faster.

2.5 AI Category Suggestion

In parallel with the grammar check, the AI service classifies the complaint text into one of the campus categories (for example Plumbing, Electrical, Housekeeping or Civil) and returns a suggested category with a confidence score and a short justification. The student can accept the suggestion or choose a different category. Accurate categorisation at source improves routing and reduces the chance of a complaint reaching the wrong department.

2.6 Duplicate-Complaint Detection

After a complaint row is created, the backend asks the AI service to compute a 768-dimensional embedding of its description and store it in the pgvector column. New complaints are compared against existing embeddings using cosine similarity; when a submission is highly similar to an earlier one it can be flagged as a probable duplicate. This prevents the queue from filling with repeated reports of the same underlying issue.

The AI service is designed to fail safe: if Gemini or the database is unavailable, each endpoint returns a safe default rather than an error, and the complaint is still created normally. AI is treated as best-effort augmentation, never a hard dependency of the core workflow.

2.7 SR Review and Assignment Workflow

Newly submitted complaints enter a PENDING_SR_REVIEW state and appear in the Student Representative's queue. The SR verifies the complaint, and either returns it to the student for clarification or approves it and assigns it to the responsible staff member or department. This human review gate keeps the queue clean and ensures complaints are correctly routed before staff are asked to act on them.

2.8 SLA Tracking and Automatic Escalation

Every complaint carries a service-level deadline based on its severity. Scheduled background jobs run periodically on the backend: one marks complaints whose deadline has passed as SLA_BREACHED so that overdue issues are visible and can be escalated, and another auto-approves complaints that have been stuck in SR review beyond a threshold so that the pipeline never stalls. SLA tracking turns vague promises of "we'll look into it" into measurable, enforced commitments.

2.9 Real-Time Push Notifications

SCMS uses Firebase Cloud Messaging to deliver push notifications for important events — a complaint being assigned, its status changing, or a resolution being recorded — even when the app is in the background. In-app banners and deep links take the user straight to the relevant complaint. This keeps every party informed without anyone having to poll the app for updates.

2.10 Offline Draft Support

Because campus connectivity is not always reliable, the app can save a complaint as a local draft using an on-device store. If the network is unavailable at submission time, the repository falls

back to the saved draft, and the complaint can be completed once connectivity returns. This ensures that a student is never blocked from at least capturing an issue.

2.11 Ratings and Analytics

After a complaint is resolved, the student can rate the resolution and leave a comment, providing a feedback signal on service quality. Administrators see aggregated analytics — complaint volumes, statuses, category distribution and SLA performance — giving management the data needed to identify recurring problems and allocate resources.

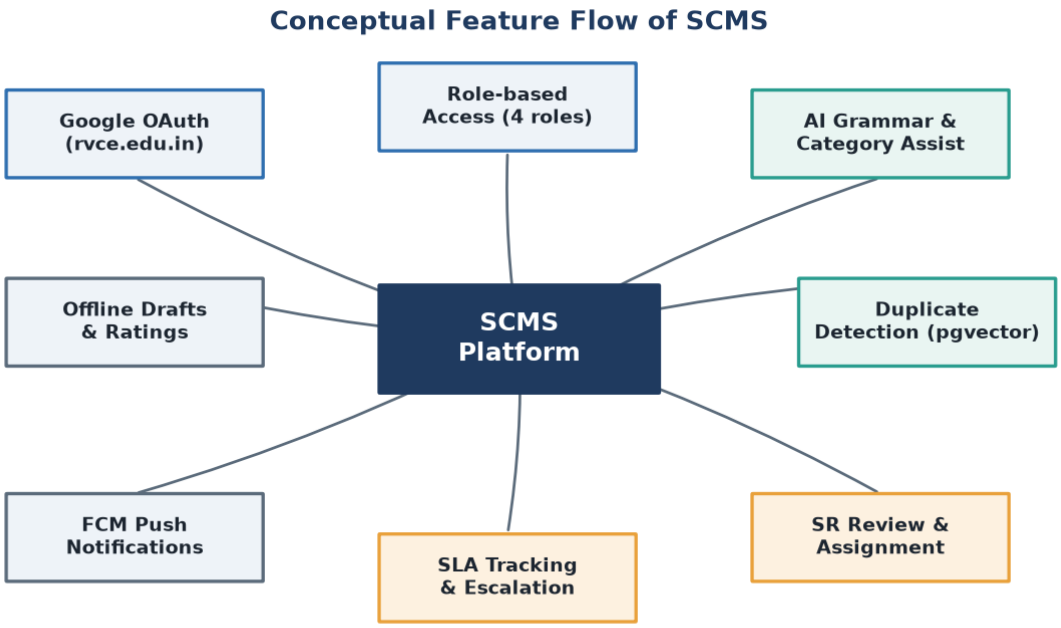

Figure 2: Conceptual feature flow of SCMS

3. Technology Stack

SCMS is built on a modern, layered stack spanning a cross-platform mobile client, a Node.js API, a Python AI microservice and a vector-capable relational database. The technologies were chosen for security, responsiveness and the ability to integrate AI without compromising the reliability of the core workflow.

3.1 Mobile Application Technologies

Flutter (Dart): a single codebase compiled to native Android (and iOS) builds, providing the entire user interface and a consistent, responsive experience with light and dark themes.

- BLoC / Cubit — predictable state management, one bloc per feature (auth, complaint, submission, SR review, analytics).

- Dio — HTTP client with interceptors that attach the JWT and transparently unwrap the backend response envelope.
- go_router — declarative routing with role-based redirects so each role sees only its permitted screens.
- Hive — lightweight on-device storage for offline complaint drafts.
- flutter_secure_storage — encrypted storage for access and refresh tokens.

3.2 Backend Technologies

Node.js with the Express framework provides the REST API on port 3000.

- Express — routing, middleware, static media serving, and a uniform success/error response envelope.
- Prisma ORM — type-safe database access and schema migrations against PostgreSQL.
- node-cron — scheduled jobs for SLA-breach marking and SR auto-approval.
- Helmet, CORS and Morgan — security headers, cross-origin control and request logging.

3.3 AI Service Technologies

A Python FastAPI microservice on port 8000 encapsulates all AI logic.

- FastAPI — fast, typed HTTP endpoints for grammar-check, categorize, embed and check-duplicate.
- google-genai (Google Gemini) — gemini-2.5-flash for text tasks and gemini-embedding-004 for 768-dimensional embeddings.
- psycopg2 with pgvector — pooled database access and cosine-similarity search for duplicate detection.

3.4 Database Technology

PostgreSQL is the primary datastore, extended with the pgvector extension. Relational tables model users, complaints, media items, status-update timelines, departments, categories, zones, tags and refresh tokens, while the vector column stores complaint-description embeddings for similarity search. Running Postgres and pgvector together in one engine keeps transactional data and vector search consistent and simple to operate.

3.5 Authentication and Security Technologies

- Google OAuth 2.0 (google-auth-library) — identity restricted to the rvce.edu.in domain.
- JSON Web Tokens — stateless access tokens and rotating refresh tokens with role claims.
- Domain allowlist — a second server-side check of the signed-in email domain.
- Helmet and HTTPS — secure headers and encrypted transport.

3.6 Development and Deployment Tools

- Docker Compose — runs the pgvector-enabled PostgreSQL container.

- Firebase (firebase-admin) — push notification delivery via Cloud Messaging.
- nodemon — backend hot-reload during development.
- Prisma Studio — GUI inspection of the database.
- Git — version control with a documented per-member work division.

4. Requirements & Specifications

4.1 Functional Requirements

- Users must authenticate through Google OAuth restricted to the rvce.edu.in domain.
- Students can create a complaint with a subject, description, severity, optional location and up to three photos.
- The system watermarks each attached photo with GPS coordinates and a timestamp.
- The app offers AI grammar correction and an AI-suggested category while composing a complaint.
- The system detects probable duplicate complaints using description embeddings.
- Newly submitted complaints enter SR review before assignment.
- SRs can approve and assign complaints to staff, or return them to the student.
- Staff can update the status of assigned complaints through to resolution.
- The system tracks an SLA deadline per complaint and marks breaches automatically.
- The system sends push notifications on assignment, status change and resolution.
- Students can rate a resolved complaint and leave a comment.
- Admins can manage users, departments, categories and zones, and view analytics.

4.2 Non-Functional Requirements

- Security — all protected endpoints require a valid JWT; roles are enforced server-side.
- Reliability — AI features fail safe; core complaint handling continues if the AI service is down.
- Performance — typing-time AI calls are debounced to avoid excessive requests.
- Usability — mobile-first, responsive UI with light and dark themes.
- Availability — offline drafts allow complaints to be captured without connectivity.
- Maintainability — layered architecture with clear module and file-ownership boundaries.
- Scalability — stateless JWT auth and a separable AI microservice.

4.3 System Specifications

Software:

- Mobile: Flutter SDK, Android (API level per device); Dart.
- Backend: Node.js with Express; Prisma; runs on port 3000.
- AI service: Python 3 with FastAPI and Uvicorn; runs on port 8000.

- Database: PostgreSQL 16 with the pgvector extension (via Docker Compose).
- External services: Google OAuth, Google Gemini API, Firebase Cloud Messaging.

Hardware (development / demo):

- An Android smartphone or emulator for the client.
- A development machine hosting the backend, AI service and database.
- Network connectivity between the phone and the backend for live use.

5. User Interface Screens

The SCMS interface follows a clean, iOS-inspired visual language with grouped sections, clear status chips and full light/dark theme support. The following screens illustrate the main student-facing flows and the AI assistance. Screens for the Staff, SR and Admin roles share the same components and are described through the workflow diagrams in Section 6.

5.1 Authentication and Dashboard

Users sign in with Google, restricted to institutional accounts, and on success are redirected to the dashboard appropriate to their role. The student dashboard summarises complaint counts, highlights items that need attention and shows a live status breakdown.

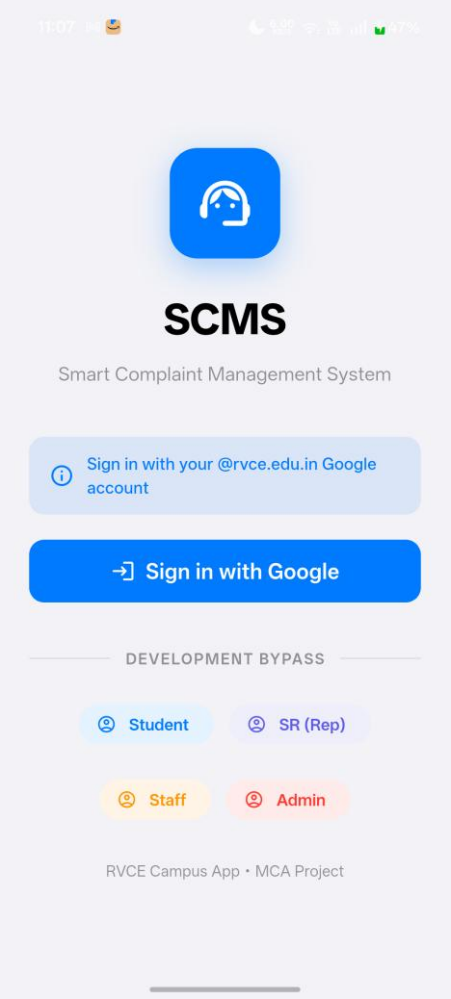

Figure 3: Login screen with Google Sign-In (restricted to rvce.edu.in)

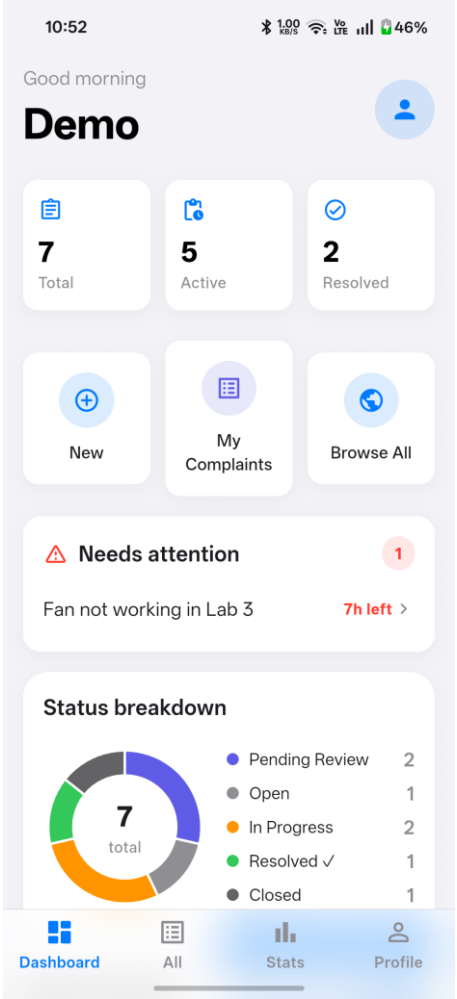

Figure 4: Student dashboard with live complaint data and status breakdown

5.2 Complaint List and Submission

The complaint list shows every complaint with a status chip, severity, category and SLA timer, and can be searched and filtered. The new-complaint form collects the subject, description, location, category and severity, and allows up to three photos to be attached.

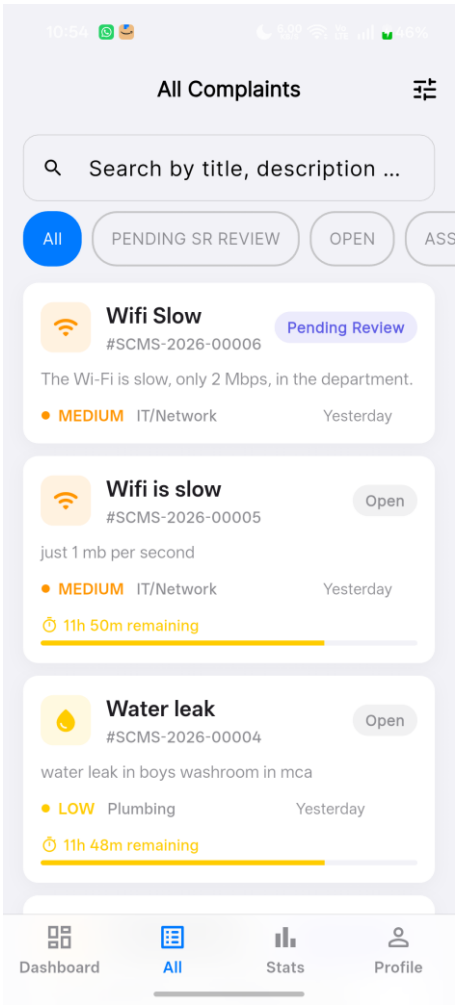

Figure 5: Complaint list showing status chips, severity and SLA timers

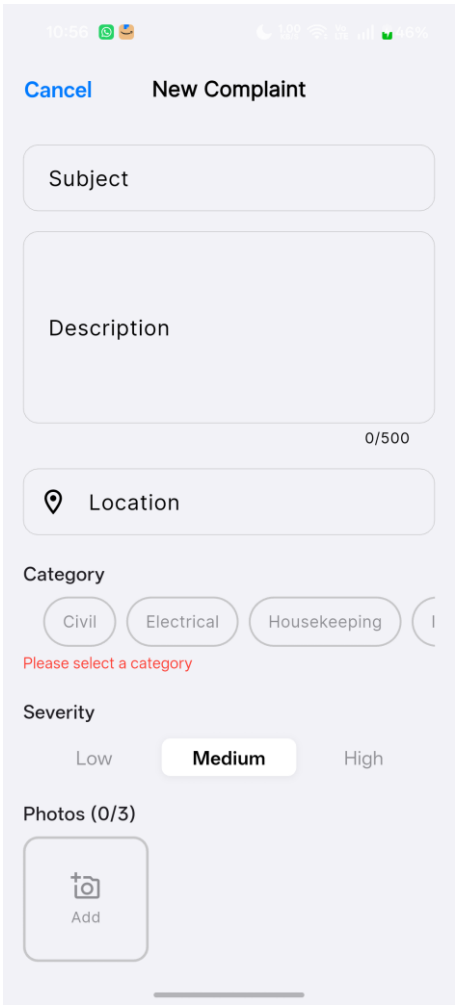

Figure 6: New-complaint form with subject, description and category chips

5.3 AI Assistance — Grammar and Category

As the description is typed, the AI grammar suggestion and the AI category suggestion appear. Figure 8 shows the corrected description applied together with the suggested category (Plumbing) at 100% confidence and a short justification.

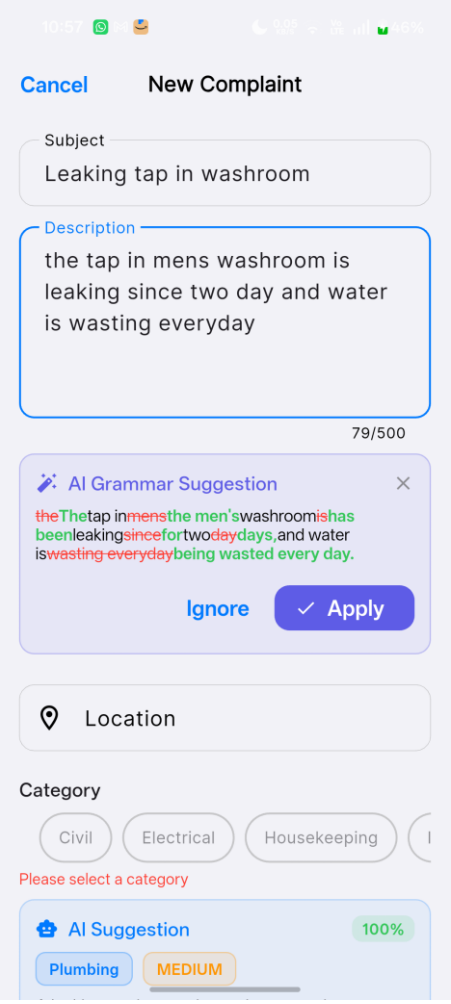

Figure 7: AI grammar suggestion and AI category suggestion while composing

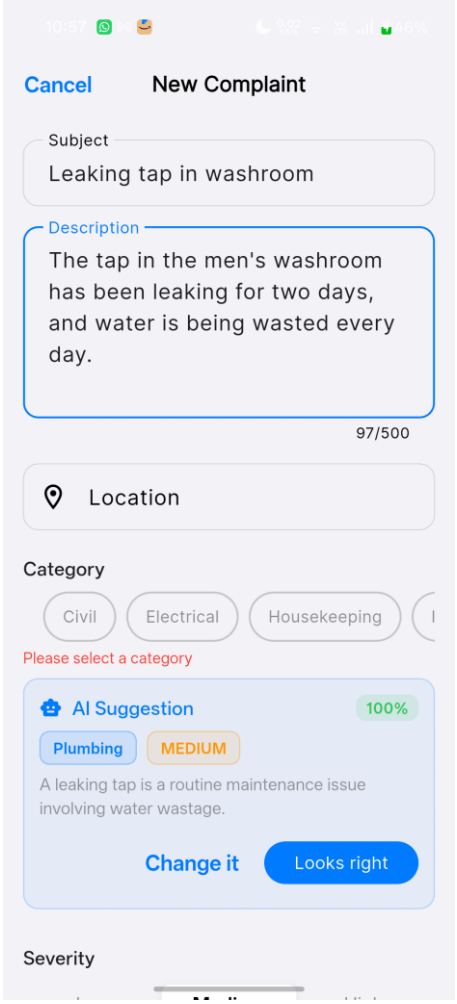

Figure 8: AI grammar correction applied and AI category suggestion (Plumbing, 100%)

5.4 Complaint Detail and Analytics

Each complaint has a detail view with its full metadata and status timeline. After submission the feed updates to reflect the new complaint, the statistics screen presents aggregated analytics, and the profile screen groups account and preference settings.

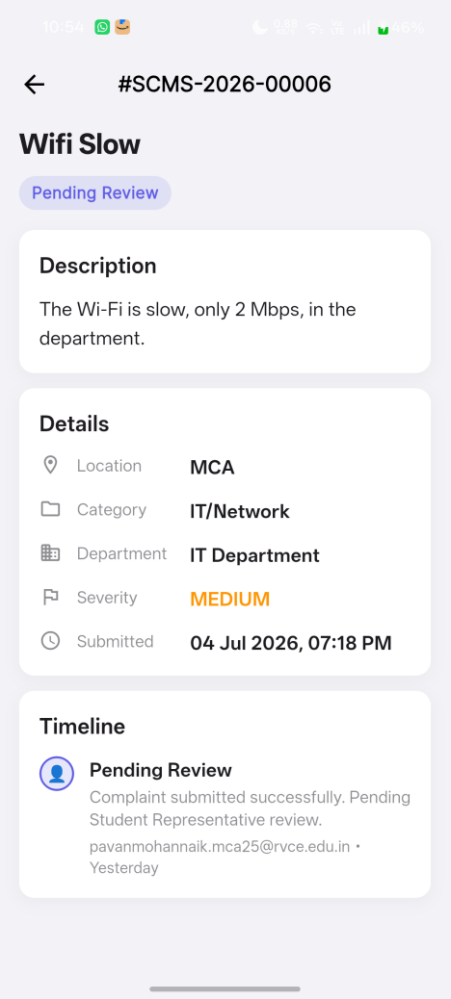

Figure 9: Complaint detail with details and status timeline

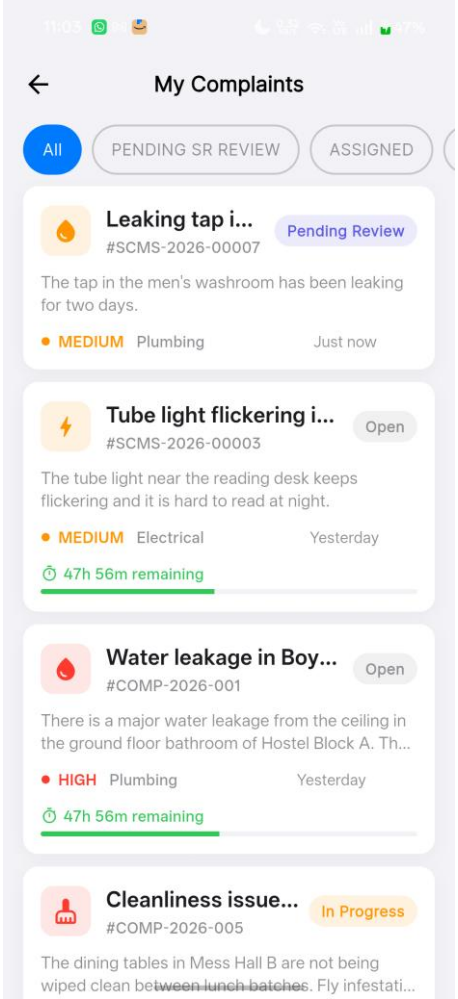

Figure 10: Updated feed after submitting a complaint (Pending Review)

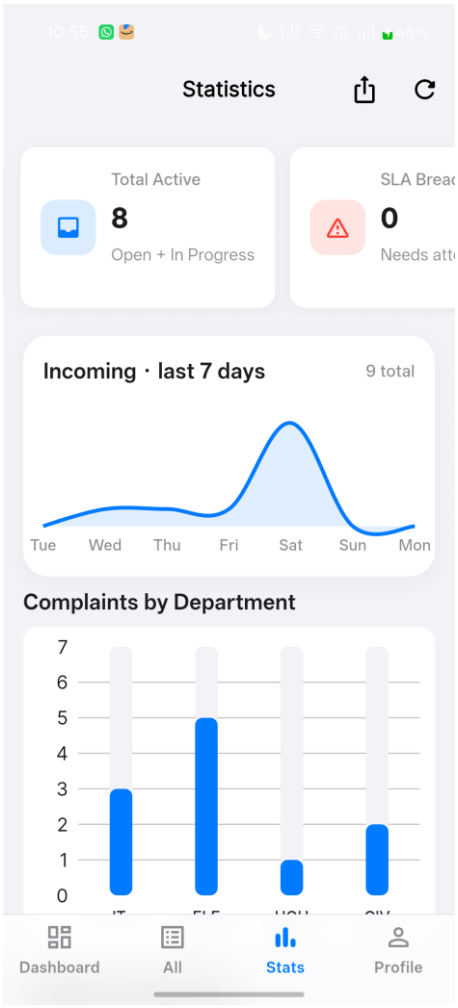

Figure 11: Statistics / analytics dashboard

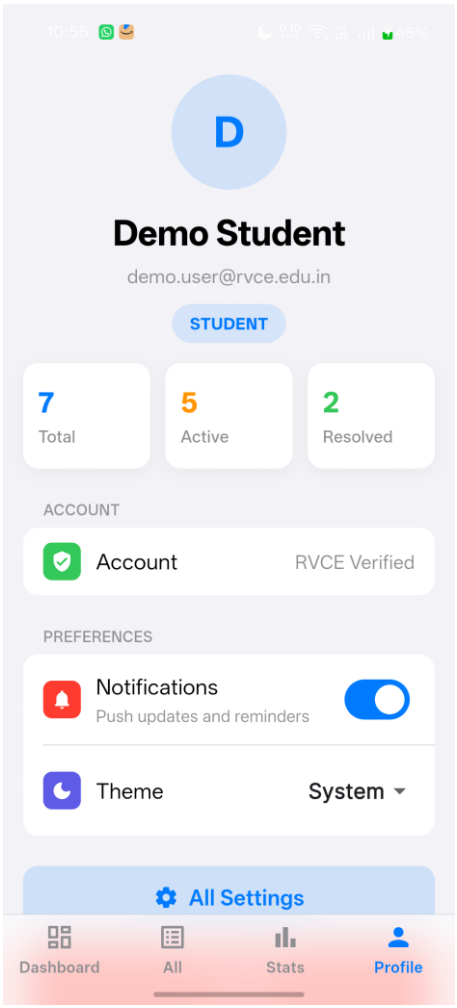

Figure 12: Profile and grouped settings screen

5.5 Navigation Flow

Navigation is driven by the authenticated role. After sign-in, go_router redirects the user to the correct role home and exposes only the routes that role is allowed to use, as shown in Figure 13.

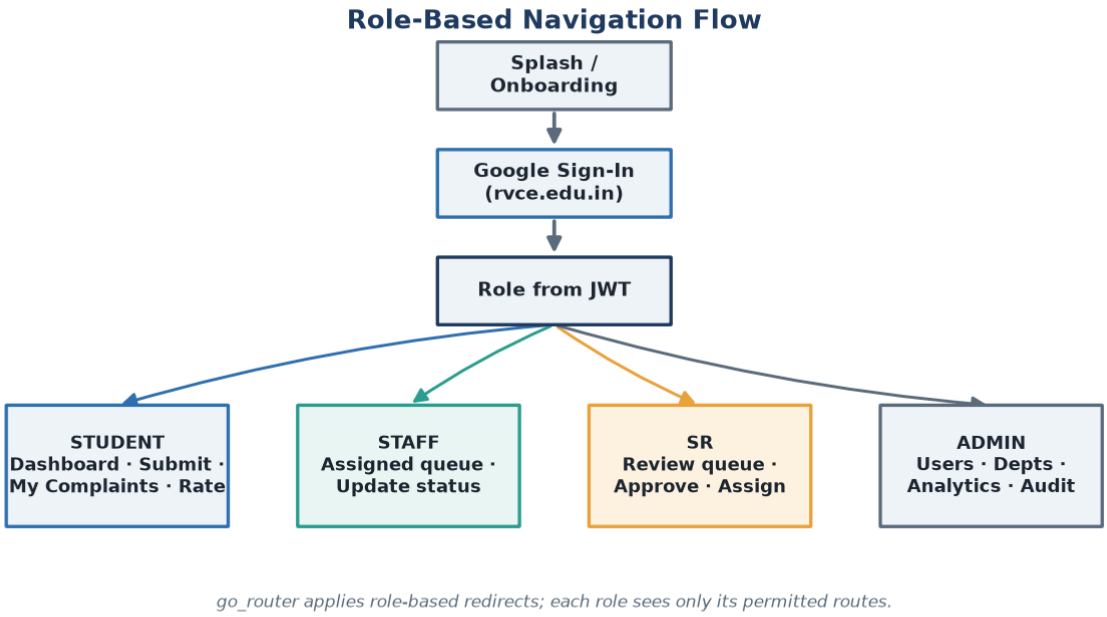

Figure 13: Role-based navigation flow of SCMS

6. Implementation Details

The system is organised into cohesive modules across the three services. Each module has a single responsibility and communicates through well-defined HTTP interfaces and the shared response envelope.

6.1 Authentication Module

The Flutter app performs Google Sign-In and sends the resulting ID token to the backend's /api/auth/google endpoint. The backend verifies the token with Google, checks the email domain against the allowlist, provisions or looks up the user, and issues its own access and refresh JWTs carrying the user's role. A Dio interceptor on the client attaches the access token to every request and refreshes it automatically on expiry.

6.2 Complaint Submission Module

Complaint creation is a multipart request carrying the subject, description, severity, optional location, JSON-encoded tags and the media files. Photos are watermarked with GPS and timestamp on the device before upload and stored by the backend's storage service. If a category is chosen, the backend resolves the responsible department from the category; complaints are created in the SUBMITTED / PENDING_SR_REVIEW state.

6.3 AI Proxy Module

All AI functionality is reached through a single backend proxy that forwards requests to the Python service's /grammar-check, /categorize, /embed and /check-duplicate endpoints. Grammar and categorisation run while the user types (debounced by 800 ms); embedding and duplicate checks run after the complaint row exists. Every AI endpoint fails safe, returning a usable default if Gemini or the database is unavailable. Figure 14 shows the request sequence.

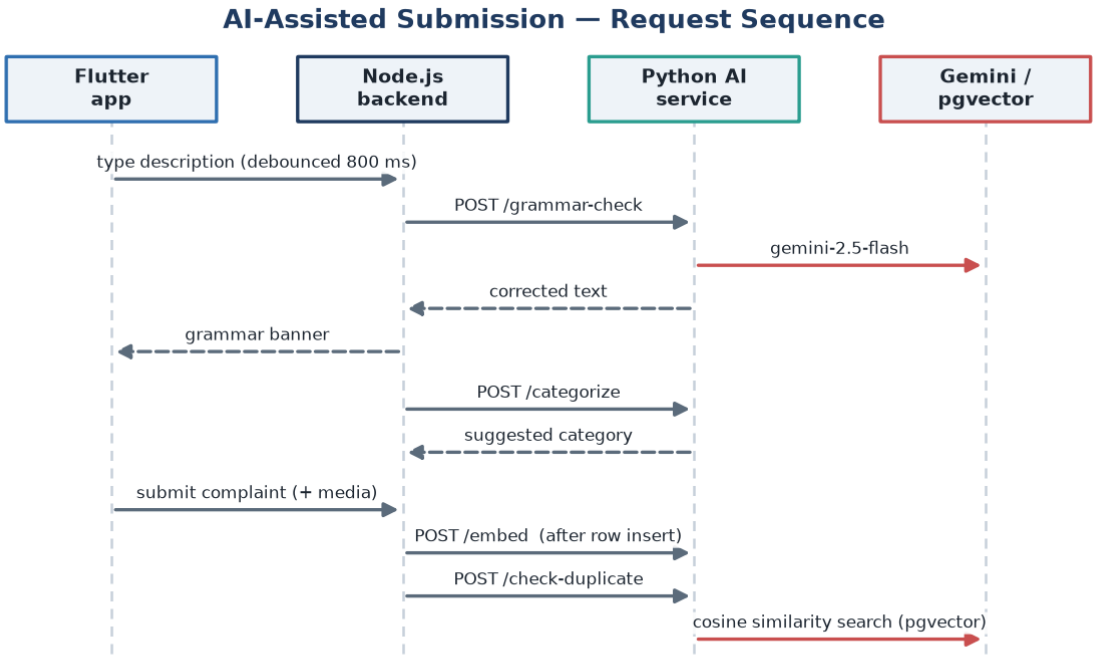

Figure 14: AI-assisted submission request sequence

6.4 SR Review and Assignment Module

The SR module lists complaints awaiting review, and lets the SR approve and assign a complaint to staff or return it to the student. On assignment the complaint moves to ASSIGNED and the assigned staff member is notified. Figure 15 shows the end-to-end review workflow across the student, AI service, SR and staff.

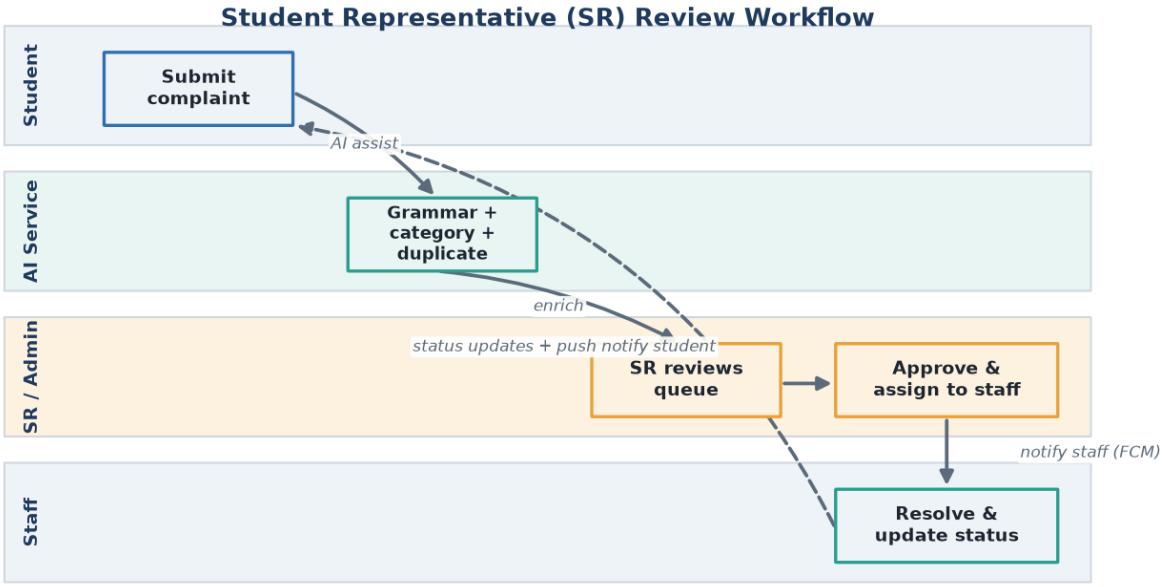

Figure 15: Student Representative (SR) review workflow

6.5 Status and SLA Module

Staff advance a complaint through IN_PROGRESS to RESOLVED, and each change is recorded on the complaint's status timeline. Two node-cron jobs support the SLA policy: one marks overdue complaints as SLA_BREACHED, and another auto-approves complaints left too long in SR review. Figure 16 shows the full status lifecycle.

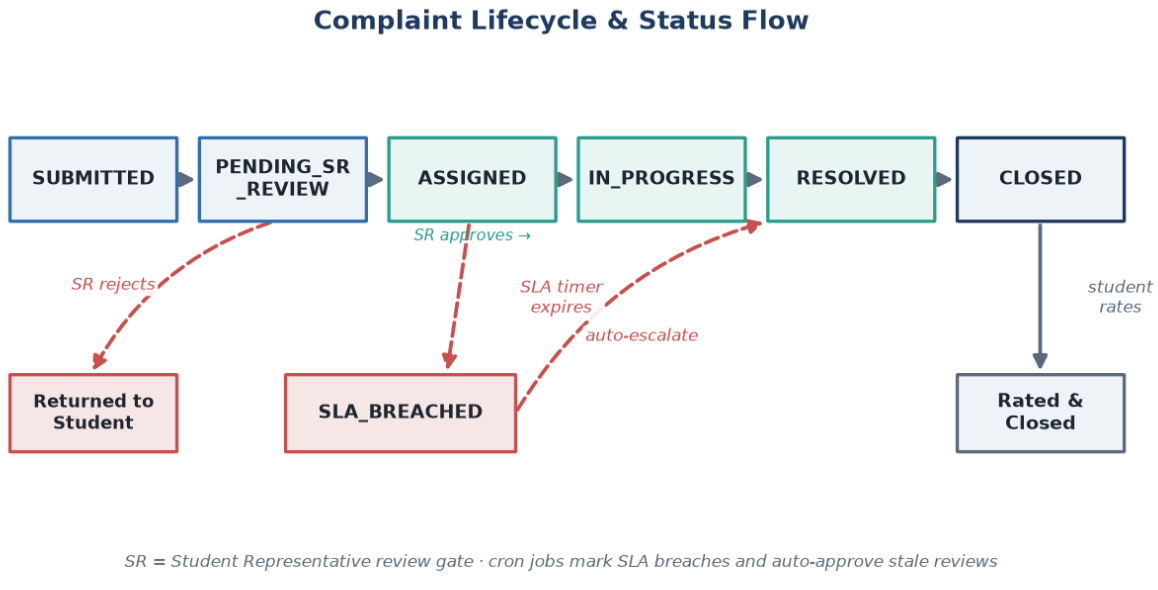

Figure 16: Complaint lifecycle and status flow

6.6 Notification Module

The backend sends push notifications through Firebase Cloud Messaging when a complaint is assigned, changes status or is resolved. On the client a notification service surfaces these as system notifications and in-app banners, and deep-links the user to the relevant complaint.

6.7 Media and Watermark Module

A dedicated watermark service stamps GPS coordinates and the capture date-time onto each photo using a custom painter before the image is uploaded. The backend stores media locally and serves it statically, and an enrichment helper attaches photo URLs and denormalised display fields to complaint responses.

6.8 Offline Draft Module

The complaint repository combines the remote data source with a local Hive store. When the network is unavailable it falls back to a saved draft, allowing a complaint to be captured offline and completed once connectivity is restored.

6.9 API and Response-Envelope Layer

Every backend response is wrapped in a uniform envelope — a success flag with a data payload, or an error object with a message and code. A client-side interceptor strips this envelope so that data sources receive the raw payload directly. This single convention keeps error handling consistent across the whole API surface.

6.10 Overall System Workflow

Bringing the modules together, a typical complaint flows as follows: the student authenticates with Google; composes a complaint assisted by AI grammar and category suggestions; submits it with watermarked photos; the backend stores it, embeds it and checks for duplicates; the SR reviews and assigns it; staff are notified and work the complaint to resolution while the SLA job watches the deadline; and finally the student is notified and rates the outcome. The architecture in Figure 1 and the workflow in Figures 15 and 16 together describe this end-to-end path.

7. Demonstration

7.1 Demonstration Description

The demonstration runs the complete stack — the Flutter app on an Android device, the Node.js backend, the Python AI service and the PostgreSQL / pgvector database — and walks a single complaint through the full lifecycle across all four roles.

7.2 Demonstration Flow

- Sign in as a student with a Google rvce.edu.in account.
- Start a new complaint; observe AI grammar correction and the AI-suggested category appear while typing.
- Attach a photo and see the GPS + date-time watermark applied.
- Submit the complaint; the backend stores it, embeds it and runs a duplicate check.
- Sign in as the SR; review the pending complaint and assign it to a staff member.
- Sign in as staff; move the complaint through In Progress to Resolved and observe the status timeline update.
- Receive push notifications on status changes; as the student, rate the resolved complaint.
- Sign in as admin; view the analytics dashboard reflecting the new activity.

7.3 Demonstration Notes

The screenshots in Section 5 are captured from live runs of the application against the running backend and AI service, including the AI grammar correction and category suggestion shown in Figure 8. Because the AI service is best-effort, the demonstration also shows the system continuing to function normally when an AI response is temporarily unavailable.

8. Conclusion

8.1 Contributions of the Project

SCMS delivers a complete, accountable complaint-management workflow for a campus, replacing informal reporting with an authenticated, role-based, SLA-tracked system. Its distinctive contribution is the pragmatic integration of AI — grammar correction, automatic categorisation and embedding-based duplicate detection — as a best-effort augmentation layer that improves submission quality and routing without ever becoming a single point of failure for the core service.

8.2 Academic Learning Outcomes

- Designing and building a cross-platform mobile app in Flutter with clean, layered architecture and BLoC state management.
- Building a secure REST API in Node.js/Express with JWT auth, Prisma and a consistent response envelope.
- Integrating a Python FastAPI microservice and a third-party LLM (Google Gemini) as a fail-safe augmentation.
- Applying vector search (pgvector) to a real problem — duplicate detection via embeddings.
- Implementing OAuth 2.0 with domain restriction, and push notifications with Firebase Cloud Messaging.

- Coordinating a four-member team with clear file-ownership boundaries and shared project documentation.

8.3 Future Scope

- A web/admin console mirroring the mobile analytics for larger-screen management.
- Richer analytics — SLA compliance trends, category heat-maps and staff workload balancing.
- Smarter routing that learns the best department/staff from historical resolutions.
- Configurable, category-specific SLA policies and escalation chains.
- In-app chat between students and staff on a complaint thread.
- Multi-institution support with per-tenant domains and departments.

9. References

[1] Flutter Documentation — https://docs.flutter.dev

[2] Dart Language — https://dart.dev

[3] Node.js Documentation — https://nodejs.org/en/docs

[4] Express Framework — https://expressjs.com

[5] Prisma ORM — https://www.prisma.io/docs

[6] PostgreSQL — https://www.postgresql.org/docs

[7] pgvector Extension — https://github.com/pgvector/pgvector

[8] FastAPI — https://fastapi.tiangolo.com

[9] Google Gemini API (google-genai) — https://ai.google.dev

[10] Google Identity / OAuth 2.0 — https://developers.google.com/identity

[11] Firebase Cloud Messaging — https://firebase.google.com/docs/cloud-messaging

[12] JSON Web Tokens — https://jwt.io